

Assignment 2-Case Study

Stage 3 Tutorial Activities

From Requirements to Domain Models

Week 6 | 60 minutes

Candidate Concepts

Candidate	Class?	Reason
Patient	Yes	Contains information to be specified such as the patient's name, phone number, date of birth, etc. Supports requirement for patient search
Practitioner	Yes	Contains information to be specified such as practitioner name, availability, etc. Supports requirement to prevent double-bookings
Appointment	Yes	Contains information to be specified such as time of appointment, practitioner involved, patient involved, status, etc. Supports requirement for appointment management
Name	No	Only describes an attribute that exists as a property for classes such as the patient and practitioner
Clinic	Yes	Acts as management, where it aggregates data from other classes such as the practitioner, appointment, etc. to further create operational reports and/or organise appointment bookings Supports requirement to produce operational reports

Database	No	Serves more as an infrastructure that stores recorded data rather than having attributes and methods. A class would interact with the database with a method
Cancellation	No	A method that would be used in relation to appointments to address cancellations rather than contain its own information and methods
Status	No	Only determines the state of an appointment using enumerations, does not contain any further attributes or methods

CRC Cards

Patient

Responsibilities	Collaborators
Knows patient information	

Practitioner

Responsibilities	Collaborators
Knows practitioner information	
Knows practitioner availability	
Knows upcoming appointments	Appointment, Patient
View patient information	Patient
Inform availability to the system	Clinic

Appointment

Responsibilities	Collaborators
Knows appointment statuses	
Knows appointment details (e.g date, time, etc.)	
Knows specific patient involved in appointment	Patient
Knows specific practitioner involved in appointment	Practitioner
Update appointment statuses	

Clinic

Responsibilities	Collaborators
Knows existing patients, practitioners and appointments in the system	Appointment, Practitioner, Patient
Saves recorded data entered into the system	Appointment, Practitioner, Patient
Create operational reports	Appointment
Create appointments	Appointment, Practitioner, Patient
Register patients into the system	Patient
Search for existing patient information	Patient
Prevent double-bookings	Appointment, Practitioner, Patient
Prevent duplicate patient records	Patient

Relationship Reasoning

Patient to Appointment: which relationship and why?

Patient to Appointment has an associative (Has-A) relationship. This is because they are two different classes that have their own lifecycles, implying that they both function independently from each other. Association is the most appropriate relationship to connect the two classes as it still allows Appointment to call to specific attributes of the Patient class without having to inherit all of Patient's attributes.

If Appointment were to inherit or have any other relationship that directly makes it rely on Patient, then if the Patient class is affected by any change or modifications, then the entire Appointment class is affected which is not desired. Additionally, if the Patient class is removed whilst pertaining a relationship with Appointment that's dependent on Patient, then the Appointment class is also removed from the system, which would further cause detrimental issues with maintaining appointment history.

Practitioner to Appointment: what multiplicity?

A practitioner (1) can have **0 to many (0..*)** appointments over time.

- Based on one practitioner's overall lifetime working at the clinic, they would start with 0 appointments and can be assigned as many appointments over time depending on how long they work at the clinic and their availability.
- **Does not imply** that a practitioner can have 0 to many simultaneous appointments at one time

Should Appointment inherit from Patient?

No, otherwise it would imply that Appointment Is-A type of Patient in which Appointment is then inheriting the existing attributes of the Patient class that it does not require. Therefore both the Patient and Appointment class should be independent of each other and allow Appointment to select specific attributes they need to call to methods.

Does Clinic need to own every object?

Yes, Clinic needs to own every object as it is responsible for saving all recorded data, therefore requires all involved objects (Patient, Practitioner, Appointment) to ensure all data is maintained and accessible for all SmartCare staff. In addition, Clinic responsibilities all revolve around class collaborators, meaning it needs to call all relevant data in the object to perform the tasks such as patient search, appointment management and so on.

AI Model Critique

Critique AI proposals: PatientManager, PractitionerManager, AppointmentManager, ClinicController, NotificationManager, ScheduleEngine.

Overall, the given proposals over-complicate the system, which does not align with client needs for the following requirement: "The organisation does not want a complex hospital information system. The first version should be a manageable application suitable for a small clinic."

AI Proposal	Critique
PatientManager	Could help organise methods in later stages in regards to managing patients, however the Clinic class is already responsible and capable of managing patient data. In this stage, separating it into another class would create complexity.
PractitionerManager	Could help organise methods in later stages in regards to managing practitioners, however the Clinic class is already responsible and capable of managing practitioner data. In this stage, separating it into another class would create complexity.
AppointmentManager	Could help organise methods in later stages in regards to managing appointments, however the Clinic class is already responsible and capable of managing appointment data. In this stage, separating it into another class would create complexity.
ClinicController	Essentially adds an extra step to the process when it is not required. Clinic is already stated to have responsibilities that can already control what data it needs to collect from other objects
NotificationManager	This proposal only addresses a method that belongs to a provisional requirement that has not yet been confirmed. In addition, no class is responsible for sending out notifications therefore this proposal is unrelated and not required at this stage.
ScheduleEngine	The system already has a class (Clinic) that addresses/prevents scheduling conflicts or any functions related to what ScheduleEngine presumably does. Essentially over-complicates the system.